DBMS PROJECT REPORT

Police Records & Criminal Management System

A centralized database solution for managing police records, criminal cases, arrests, evidence and court proceedings

Submitted by

Arpit Karna (25BCE2887)

G Nikhil Sreenivas (25BCE0178)

Aryan Menon (25BCE0327)

Thota Manoj Sai Kumar (25BAI0021)

Contents

Abstract 1
Chapter 1: Introduction 1
Chapter 2: System Design 1
Chapter 3: Database Design 3
Chapter 4: Implementation 6
Chapter 5: User Interface 7
Chapter 6: Testing 8
Chapter 7: Results and Future Scope 9
References 9

Abstract

Manual, paper-based record keeping in police stations causes duplicated records, slow searching, weak case tracking and time-consuming reporting. The Police Records & Criminal Management System (PCRMS) replaces it with a centralized relational database and a role-based web application. The database is a thirteen-table MySQL 8 schema in third normal form with keys, constraints, three views, three stored procedures and four triggers; the triggers keep case and criminal statuses consistent and log every status change. A Node.js/Express API authenticates users with JWT and enforces three roles, and a React interface provides a dashboard, nine record-management modules, search and reports. The system was verified with 44 automated API and database tests and 24 user-interface tests, all of which passed.

Chapter 1: Introduction

1.1 Background and Problem

Police work produces many related records: stations, officers, complaints, FIRs and cases, criminals, arrests, evidence and court hearings. The project review document identifies four problems with manual record keeping: **data duplication**, **difficult searching**, **poor case tracking** and **time-consuming management**. A relational DBMS [5] is a natural fit because the records are tightly related and integrity matters.

1.2 Objectives and Scope

- Centralize police records in one secure, normalized database with enforced keys.
- Manage criminals, cases, arrests and evidence digitally, including an evidence chain of custody.
- Track court proceedings and produce reports instantly; support fast search and cross-referencing.
- Automate the case lifecycle inside the database with procedures and triggers.
- Restrict actions by role: viewer (read only), officer (no delete), administrator (full access).

Out of scope: user-registration screens (three demo accounts come from the setup script), photo upload (a photo URL is stored), mobile app, cloud deployment and biometric integration.

Chapter 2: System Design

2.1 Architecture

PCRMS is a three-tier application (Figure 1). The React 18 single-page application [1] talks to a Node.js/Express REST API [2] using JSON and a JWT bearer token. The API runs parameterized SQL against MySQL 8.4 [3]. Rules that must hold for every client (status changes, custody log, audit log) live in the database as procedures and triggers.

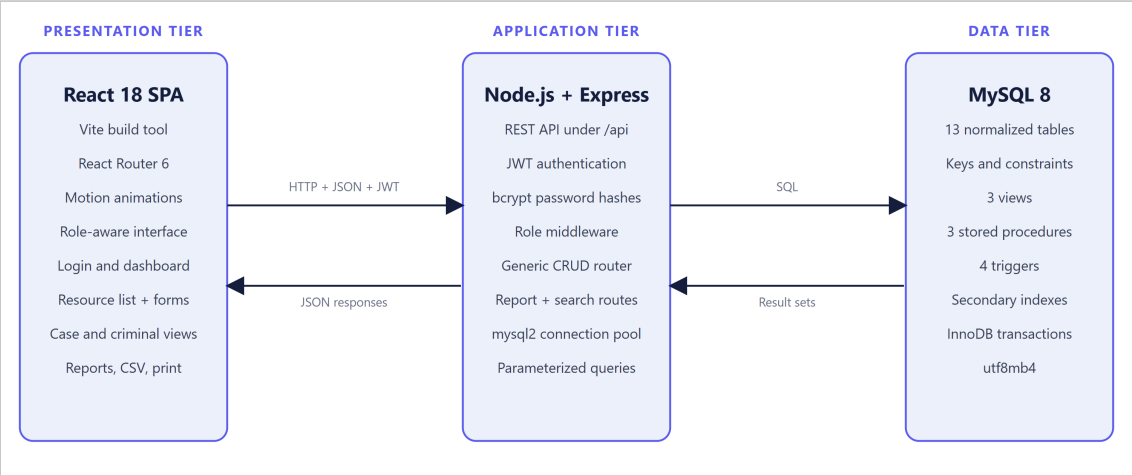


Figure 1: Three-tier architecture of PCRMS

2.2 Case Lifecycle

Figure 2 shows the main workflow. A complaint becomes an FIR (a case); an arrest marks the criminal as Arrested and the case as Under Investigation; evidence gets a custody log; a hearing moves the case to In Court; closing the case completes the lifecycle.

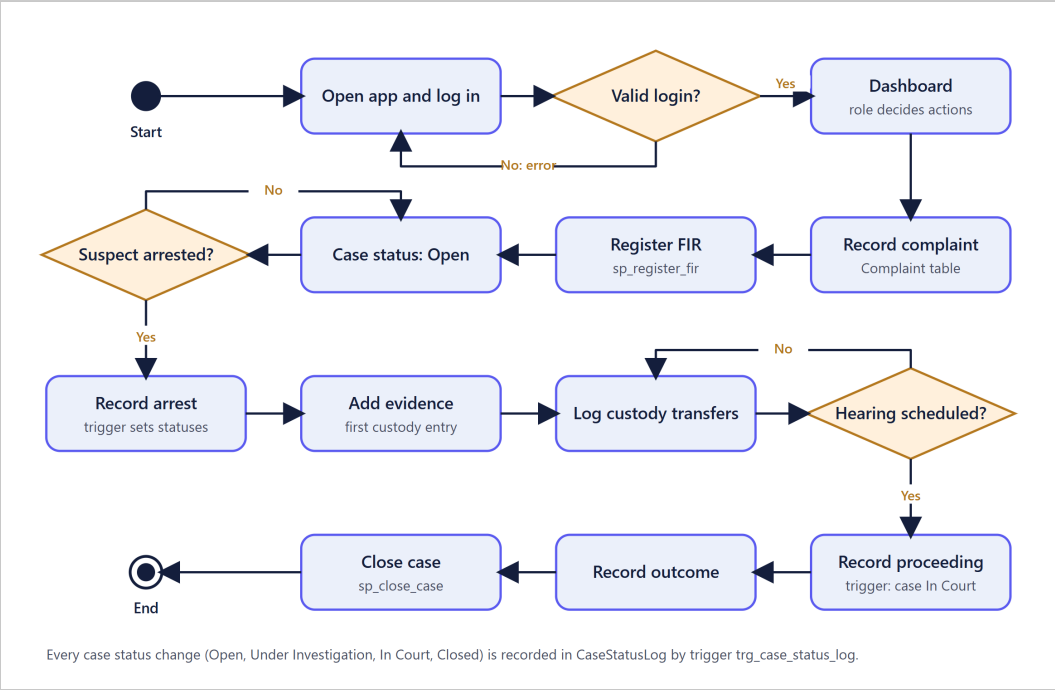


Figure 2: Case lifecycle workflow

Chapter 3: Database Design

3.1 Entity-Relationship Diagram

Figure 3 is the ER diagram in Chen notation [6]: rectangles are entities, diamonds are relationships, ellipses are attributes (underlined = primary key). Double lines mean total participation and arrows point to the "one" side of many-to-one relationships. Foreign keys appear as relationships. The many-to-many relationship **involves** is stored as the table CriminalCase, so the 12 entities and this relationship give the 13 tables. The diagram was checked against the live schema.

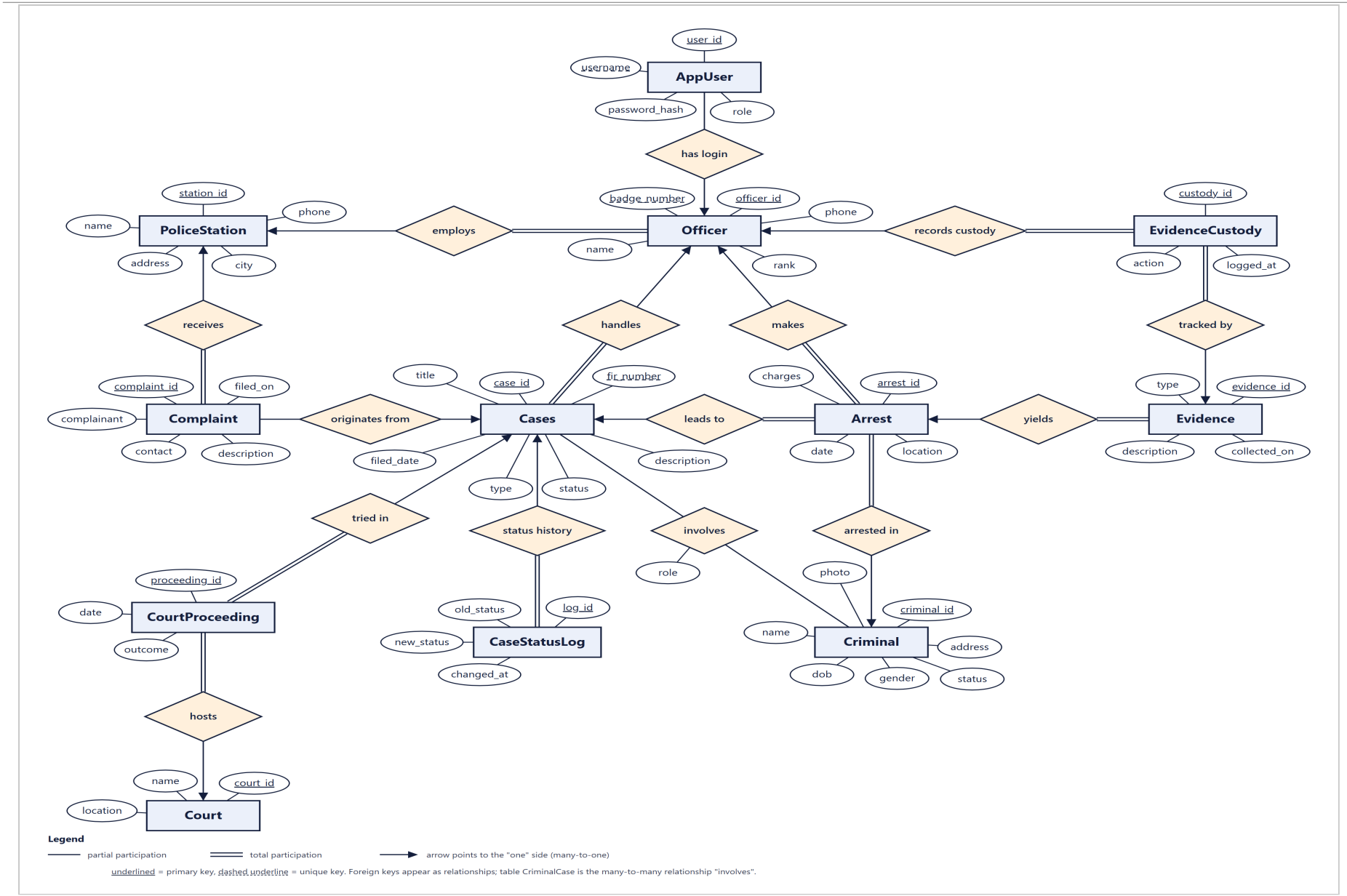


Figure 3: Entity-relationship diagram of the PCRMS database (Chen notation)

3.2 Tables and Relationships

Table 1: Tables of the pcrms database

Table	Purpose
PoliceStation	Police stations with address, city and phone. Parent of officers and complaints.
Officer	Police officers with unique badge number, rank and the station they belong to.
Criminal	Persons of interest with personal details, photo link and a status (Wanted, Arrested, Convicted, Released).
Cases	Cases created from an FIR. Holds FIR number, type, status and the officer in charge. Named Cases because CASE is a reserved SQL word.
Complaint	Citizen complaints received at a station; optionally linked to the case they led to.
CriminalCase	Junction table that resolves the many-to-many relationship between criminals and cases; stores the person's role in the case.
Arrest	An arrest of a criminal in connection with a case by an officer, with date, location and charges.
Evidence	Evidence items collected for an arrest.
EvidenceCustody	Chain-of-custody log: each hand-over or action on an evidence item, with the officer and timestamp.
Court	Courts with name and location.
CourtProceeding	Hearings of a case in a court, with date and outcome (NULL until concluded).
AppUser	Application logins with bcrypt password hash and role (admin, officer, viewer); optionally linked to an officer.
CaseStatusLog	Audit rows written by a trigger whenever a case's status changes.

The schema has 16 foreign keys. Main relationships: a station has many officers and complaints; an officer handles cases and makes arrests; a case has many arrests, hearings, status-log rows and optional complaints; an arrest has many evidence items and each item has a custody log; a court hosts many hearings. Criminals and cases form a many-to-many relationship resolved by **CriminalCase** with composite key (criminal_id, case_id). Deleting a case cascades to its links and status log but only sets Complaint.case_id to NULL, so complaints are never lost.

3.3 Normalization

- **1NF:** all columns are atomic; repeating data (criminals per case, evidence per arrest, custody actions) is stored as rows in separate tables.
- **2NF:** single-column keys cannot have partial dependencies; in CriminalCase the only non-key attribute (role) depends on the whole key.
- **3NF:** no transitive dependencies: station, court and officer details are stored once and referenced by key; counts are computed by views, not stored.

3.4 Views, Procedures and Triggers

Table 2: Programmability objects

Object	Purpose
v_case_overview, v_wanted_criminals, v_station_stats	Views used by reports and the dashboard.

Object	Purpose
sp_register_fir	Creates a case and links a complaint in one transaction; rolls back on error.
sp_close_case, sp_criminal_history	Close a case; list a criminal's cases.
trg_arrest_after_insert	Criminal becomes Arrested; Open case becomes Under Investigation; criminal linked to the case.
trg_proceeding_after_insert	Case becomes In Court when a hearing is recorded.
trg_case_status_log	Writes a CaseStatusLog row whenever a case status changes.
trg_evidence_custody_init	Creates the first chain-of-custody entry for new evidence.

Chapter 4: Implementation

4.1 Technology Stack

Table 3: Technologies and versions used

Layer	Technology
Database	MySQL 8.4.9 (InnoDB, utf8mb4)
Backend	Node.js 22, Express 4.22, mysql2 3.24, jsonwebtoken 9.0, bcryptjs 2.4
Frontend	React 18.3, React Router 6.30, Vite 5.4, Motion for React 14

The application has about 1684 lines of code in three folders: database (schema, procedures and triggers, seed data), server (API) and client (user interface).

4.2 API and Security

The API exposes 57 routes: login, dashboard, reports, FIR registration, case detail and closing, criminal history, custody log, search and generated CRUD routes for nine resources (generated from one configuration file, which also drives one generic list/form page in the interface). Passwords are stored as bcrypt hashes; a successful login returns a JWT [4] valid for eight hours. Every request is checked for a valid token and then for the role, as shown below. All SQL uses parameters and only whitelisted columns are written.

Table 4: Permissions by role

Role	Read (GET)	Create / update	Delete
viewer	Allowed	Denied (403)	Denied (403)
officer	Allowed	Allowed	Denied (403)
admin	Allowed	Allowed	Allowed

4.3 Error Handling

Database constraint errors are mapped to responses: duplicate keys and foreign-key violations return 409, missing or invalid tokens 401, forbidden roles 403. Other database errors (for example a missing required column) currently return 500 with the database message, which is a known limitation.

Chapter 5: User Interface

The screenshots below were captured from the running application with the seeded sample data. Figure 4 is the entry point; Figure 5 the dashboard; Figure 6 a case detail page that combines data from six tables; Figure 7 a data-entry form whose Criminal, Case and Officer fields are dropdowns filled from the database; and Figure 8 the same Cases page on a phone-sized screen.

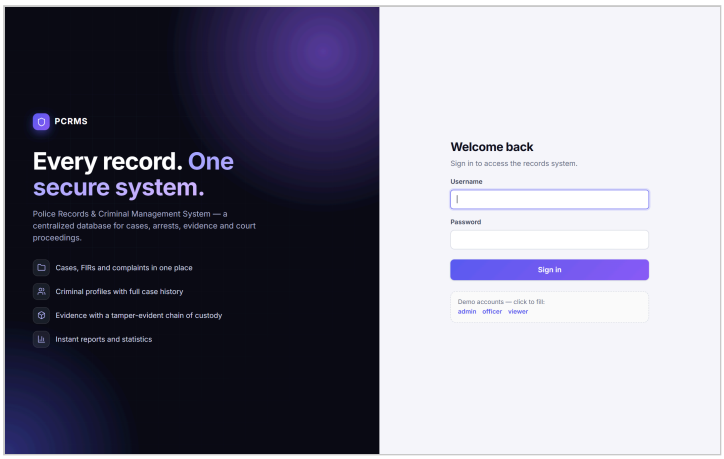


Figure 4: Login screen

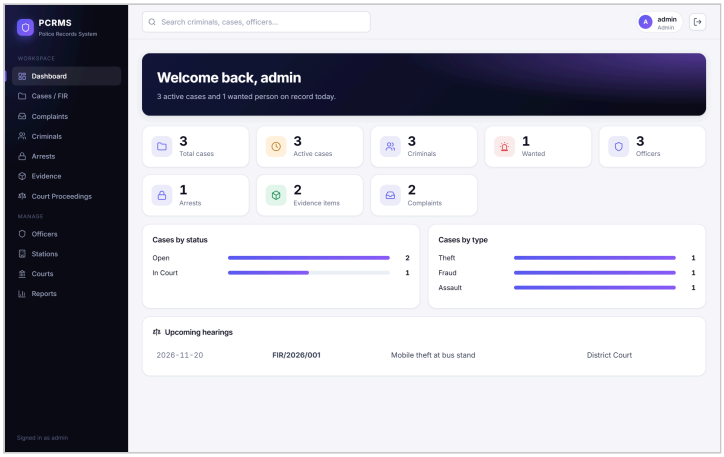


Figure 5: Dashboard

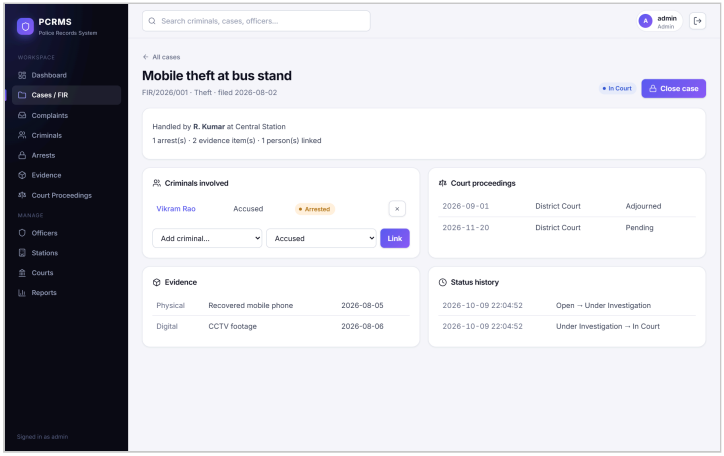


Figure 6: Case detail page

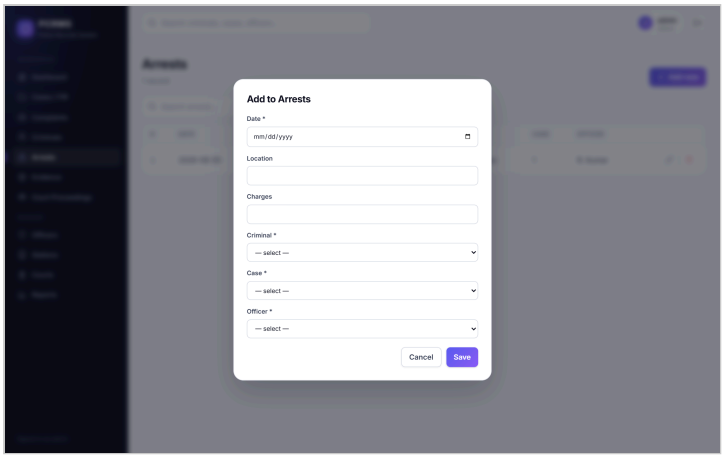


Figure 7: Add-arrest form with database-backed dropdowns

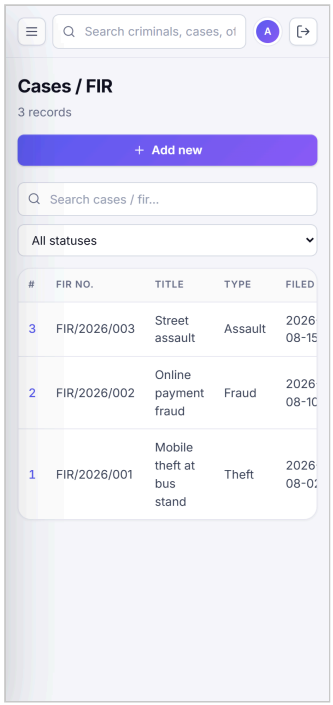


Figure 8: Cases list on a 390 px wide screen

Chapter 6: Testing

Testing used an automated runner for the API and database (44 cases, server/tests/run-tests.js) and an automated headless-Chrome session for the user interface (24 cases), on Windows 11 with Node.js 22.19 and MySQL 8.4.9. The database was reset to the seed data before the run. All 68 of 68 cases passed.

Table 5: Summary of executed tests

Group	Cases	Passed	Covers
API and database	44	44	Authentication, roles, CRUD, validation, procedures, triggers, constraints, reports, search
User interface	24	24	Login, every page, forms, filter, search, error and empty states, viewer role, mobile layout

Group	Cases	Passed	Covers
Total	68	68	

Table 6: Selected test cases (full list in the detailed report)

ID	Test	Input	Actual result
TC-01	Valid login	admin / correct password	HTTP 200, token issued, role=admin
TC-08	Viewer cannot create	POST /api/courts	HTTP 403 "Your role is not allowed"
TC-10	Officer cannot delete	DELETE /api/courts/:id	HTTP 403, record still present
TC-20	Duplicate unique FIR number	POST /api/cases with fir_number FIR/2026/001	HTTP 409 (duplicate entry)
TC-22	Cannot delete referenced parent row	DELETE /api/officers/1	HTTP 409, officer still present
TC-25	sp_register_fir rolls back on error	POST /api/fir with duplicate FIR number	HTTP 409, case count unchanged (4)
TC-26	Arrest trigger updates criminal and case	POST /api/arrests {criminal 2, case 2}	criminal 2 = Arrested, case 2 = Under Investigation
TC-31	Court proceeding moves case to In Court	POST /api/proceedings for case 3	case 3 = In Court
TC-28	Status change logged	Query CaseStatusLog for case 2	Log row: Open -> Under Investigation
TC-23	SQL injection attempt in search	GET /api/search?q=' OR 1=1; DROP TABLE Criminal;--	HTTP 200, Criminal table intact (3 rows)

Chapter 7: Results and Future Scope

The application meets the objectives: records are centralized with enforced integrity, the case lifecycle is automated and audited, evidence has a custody trail, search and reports are instant, and access is role-based. Known limitations: invalid input returns HTTP 500 instead of 400; there is no user-management or registration screen; the JWT is kept in browser storage; lists are not paginated; and there are no unit, load or cross-browser tests.

Future work (not implemented): field-level validation messages, user administration, pagination, a per-user audit trail, photo upload, per-station data access, a mobile app, a cloud database and real-time notifications.

References

1. React Team, "React Reference Documentation." [Online]. Available: https://react.dev
2. OpenJS Foundation, "Express 4.x API Reference." [Online]. Available: https://expressjs.com/en/4x/api.html
3. Oracle Corporation, "MySQL 8.4 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/
4. M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519
5. E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970.
6. P. P. Chen, "The entity-relationship model: toward a unified view of data," ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9-36, 1976.